



ELEX 7660: Digital System Design

Final Project Report

Laser Data Transmission & Tracking System

Prepared for Eduardo Casas, Instructor

Rafael Banalan

Rodriguez Chan

Table of Contents

Abstract.....	4
1 Introduction	5
2 Theory of Operation	6
2.1 Transmitter.....	6
2.2 Receiver.....	7
2.3 Quadrant Photodiode (QPD).....	8
2.4 Infrared Data Transmission	9
2.5 Search Algorithm	13
3 Challenges	14
3.1 Original Design	14
3.2 Current Design.....	15
4 Results and Analysis.....	17
5 Conclusion.....	19
6 Appendix.....	20
6.1 Infrared Transmission.....	20
6.2 Infrared Receiver Mechanics	21
6.3 Bonus photos.....	22
6.4 Transmitter Code	24
6.4.1 laser_top.sv.....	24
6.4.2 ir_rx.sv.....	26
6.4.3 search_controller.sv.....	29
6.4.4 driver.sv.....	32
6.5 Receiver Code	33
6.5.1 laser_rx.sv	33
6.5.2 adc_qpd_scanner.sv.....	34
6.5.3 rx_state.sv	36
6.5.4 qpd_state.sv.....	37
6.5.5 ir_tx.sv	39
7 References.....	43

Table of Figures

Figure 1 : Transmitter high level state machine flowchart	6
Figure 2 : Transmitter as modeled.....	7
Figure 3 : Transmitter as built.....	7
Figure 4 : Receiver high level state machine flowchart	7
Figure 5 : Inside of receiver assembly	8
Figure 6 : Quadrant photodiode arrangement	8
Figure 7 : Quadrant photodiode measurement circuit for one channel	9
Figure 8 : Example of baseband signal using the standard NEC protocol	10
Figure 9 : Example of modulated signal using the standard NEC protocol	10
Figure 10 : Example of demodulated signal from TSOP1838 module	10
Figure 11 : IR transmitter module state machine flowchart.....	11
Figure 12 : IR receiver module state machine flowchart	12
Figure 13 : Search algorithm implementation.....	13
Figure 14 : Original design as presented in project proposal.....	14
Figure 15 : Original transmitter assembly	15
Figure 16 : Close-up of baffle.....	15
Figure 17 : Impact of Distance on Tracking Stability Graphed	16
Figure 18 : Receiver during alignment	17
Figure 19 : Example of debug console during testing (receiver side)	18
Figure 20 : Close-up of transmitter head.....	22
Figure 21 : Motor drivers used for the yaw and pitch steppers	22
Figure 22 : Original transmitter gimbal robot.....	23
Figure 23 : QPD board planned on KiCad	23

Table of Tables

Table 1 : Custom IR protocol timings.....	20
---	----

Abstract

As part of our ELEX 7660 Digital System Design course, we were to complete a final project based on the provided Terasic DE0-Nano-SoC FPGA development board. The project was primarily centered around digital hardware design rather than software for implementation.

The project we chose was a laser transmission and tracking system which leverages the parallelism and deterministic timing capabilities of the FPGA to handle multiple channels of input in a predictable manner.

Our final implementation of this project exclusively uses digital hardware design in its operation, with software only used during the debugging process of our development.

As we primarily focused on the tracking feature, we could not complete the laser data transmission aspect of this project but did achieve the rest of our goals. The final implementation features hardware ADC sampling, fixed-point calculations, stepper driver addressing, and IR transmission. All modules of this project were developed independently, with the exception of the IP used in pushing the data out to UART during debugging.

All code used in this project as well as a video demonstrating its operation can be found on our [GitHub repository](#).

1 Introduction

Our project is a data transmission system designed to utilize a high-speed laser link for payload delivery. The system consists of a dual-axis gimbal-mounted transmitter and a mobile handheld receiver. To ensure ideal data transmission, the transmitter will lock onto the receiver and track its movement to keep the laser in the line of sight of the receiving photodiode.

As per our instructor, the project is assigned the following objectives:

- ✓ Azimuth (yaw) and elevation (pitch) tracking of a moving retroreflective target using a laser
- ✓ Effective tracking/transmission with the target further than 2m and of size less than 10cm

In addition, a few stretch goals were assigned:

- Target acquisition
- Data communication with the target
- Able to acquire/track at long range
- Able to track target moving at high-speed

Since the primary mode of data transmission in this project is laser, the biggest challenge of this system is knowing where the laser is pointing relative to the receiver. This is needed not only to locate the receiver, but also to center the laser on the sensor receiving the data where it will be most effective.

To achieve these goals, we use a makeshift duplex system, where the high bandwidth payload is transmitted over laser to the transmitter and the low bandwidth positional data is sent by the receiver over infrared to the transmitter. This reduces our need to depend on optics and makes the project less susceptible to component tolerances.

2 Theory of Operation

In normal operation, the transmitter looks for the receiver by scanning the laser in its search area. Since the laser is highly directional, a quadrant photodiode (QPD) on the receiver is used to determine where to aim the laser and to receive the incoming data at the same time. When the laser hits the QPD, a message is sent to the transmitter via infrared to confirm that the receiver is in range and can start locking into the center of the QPD. To do so, the receiver tells the transmitter where the laser is hitting on the QPD, and the transmitter corrects for that in order to maintain its aim on the center.

All logic in this project was done on hardware, with software being used only in the debugging process. Since the software capabilities of the SoC were not used in the core functionality of the project, it has been omitted from this report.

2.1 Transmitter

The transmitter is responsible for aiming the laser at the receiver. It does this by moving between three states: an idle state where it waits for the receiver to see the laser, an active state where it uses data from the receiver to position the laser, or a search state where it actively searches for the receiver. A simplified explanation as to how this high-level state machine behaves can be seen in the flowchart below.

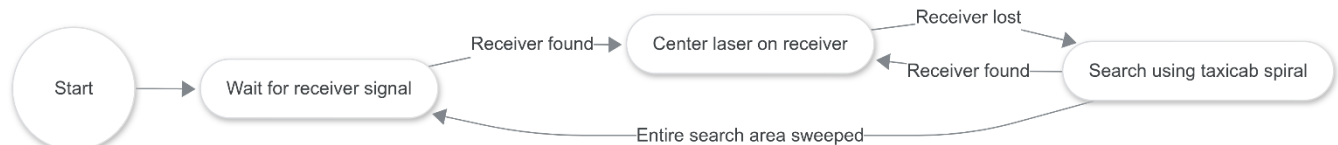


Figure 1 : Transmitter high level state machine flowchart

The transmitter is designed as a dual-axis gimbal system that tracks the receiver's position. The assembly consists of three primary components: yaw and pitch stepper motors, a laser diode, and an IR receiver. These components are connected on the FPGA. To achieve a full range of motion, the laser and IR receiver are mounted on an arm; the pitch stepper controls the vertical tilt, while the yaw stepper manages the horizontal rotation. This configuration allows the system to dynamically align the laser with the target.

To determine if the receiver has been found, it listens for a signal using the IR receiver module. When the laser is not hitting the receiver, no IR signal is being transmitted, telling the transmitter to start looking until it times out.

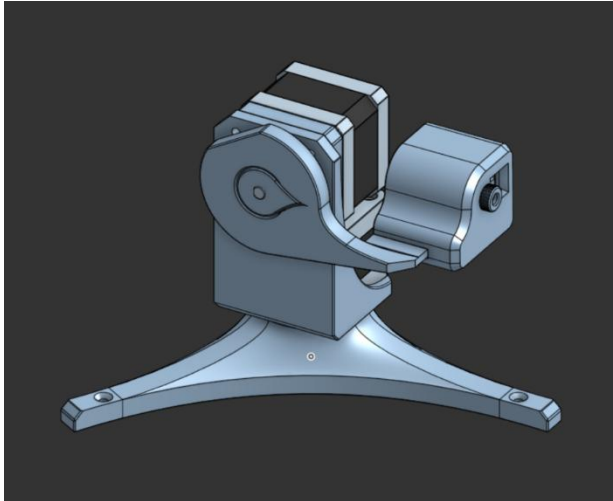


Figure 2 : Transmitter as modeled



Figure 3 : Transmitter as built

The transmitter's positioning is driven by two NEMA-17 stepper motors interfaced with A4988 drivers, configured for 1/16th microstepping to achieve an angular resolution of 0.1125° per step. We used a 5mW laser diode for tracking and a TSOP1838 IR receiver module, which has an internal demodulator—useful for decoding.

2.2 Receiver

The receiver is responsible for telling the transmitter where the laser is relative to its center. When the laser is in range, it hits a Quadrant Photodiode (discussed in 2.3) located on the face of the receiver to calculate the laser's error relative to the center. This data is sent via IR back to the transmitter to tell it which direction to move to correct for the error.

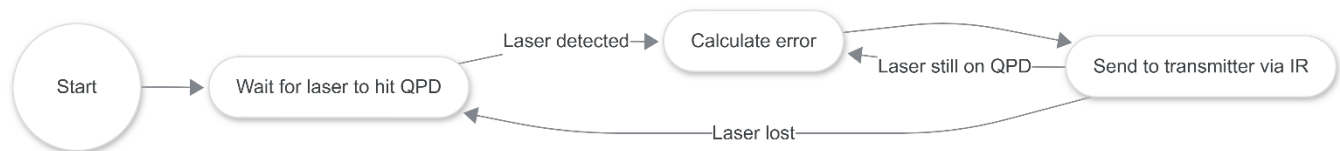


Figure 4 : Receiver high level state machine flowchart

For the purposes of this project, the receiver is designed to be held in the hand so it can be moved freely relative to the laser. It consists of the QPD system and the IR transmitter, both of which can be found on a custom made protoboard placed inside a housing. This module is separate from the transmitter and uses its own FPGA. The offset and IR signal are both generated independently from the transmitter.



Figure 5 : Inside of receiver assembly

2.3 Quadrant Photodiode (QPD)

A quadrant photodiode (QPD) is a chip that holds four photodiodes arranged in a square on a single piece of silicon. They are used to determine the position of a beam of light by calculating the difference in intensity across its four quadrants ^[1]. Ideally, a real quadrant photodiode would have been used in our project, but they are unfortunately very expensive.

To combat the added cost, we came up with a makeshift solution: we use four standard photodiodes (in this case the BPW34) and arrange them in quadrature (see Figure 6) to mimic a real QPD. The photodiodes are then connected to a transimpedance amplifier to map the changes in current to changes in voltage so the results can be easily read via the onboard ADC. The op-amps used are rail-to-rail to maximize and limit the voltage to the reference voltage of the ADC.

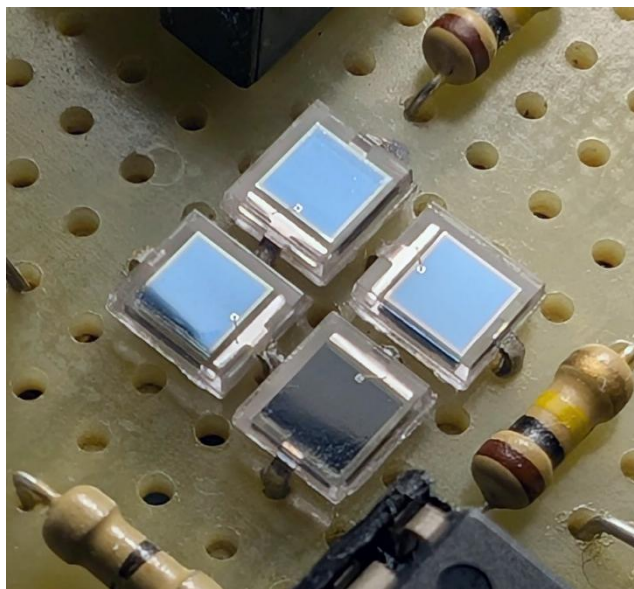


Figure 6 : Quadrant photodiode arrangement

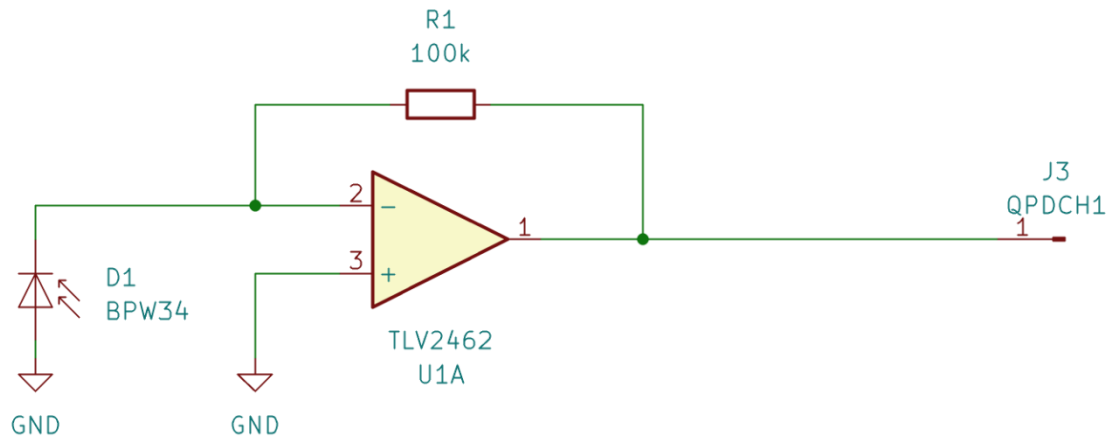


Figure 7 : Quadrant photodiode measurement circuit for one channel

This solution proves to be precise enough to correct for minute errors in position. Even though the main advantage of using a real QPD is the identical gains between the four channels, we minimize this error in the makeshift QPD by normalizing the channel values in processing. The result is a mildly noisy but distinct profile with an acceptable signal-to-noise ratio which is reliable enough to correct the position of the transmitter.

2.4 Infrared Data Transmission

In order to transmit the positional data back to the laser transmitter, we use infrared as a secondary communication channel. In real laser data transmission systems, radio is typically used as the secondary channel for alignment.

We are using a modified version of the NEC infrared protocol^[2] for our IR transmission. The main difference between the NEC standard and the one used here is that NEC uses an 8-bit address and an 8-bit command message. Ours uses a simple 8-bit data message. This simplification is made both to shorten transmission time and avoid sending eight extra unnecessary bits in transmission. The address data was traditionally used to address different devices (since it is a TV remote standard) and prevent confusion regarding which device is supposed to listen. Additionally, our protocol uses much shorter timings than the original NEC standard. This is to maximize the refresh rate of the laser alignment without violating the timing minimums imposed by the IR receiver module.

Transmission is done in bursts of five packets with a longer wait after so as not to trigger the automatic gain control of the TSOP1838 IR receiver module^[3]. Without this wait, the module will start to see the signal as part of the ambient noise in the room and filter it out. The baseband signal is generated using a state machine, which is ANDed with a clock divided 38 kHz carrier.

Receiving the signal is done on the FPGA as well. We use a state machine with a timer. The timer counts the number of ticks between states and sees if the count is within a threshold. If it is, the state is counted as valid, and the state machine advances. A full description of the receiver module can be found in the appendix¹.

¹ See Infrared Receiver Mechanics

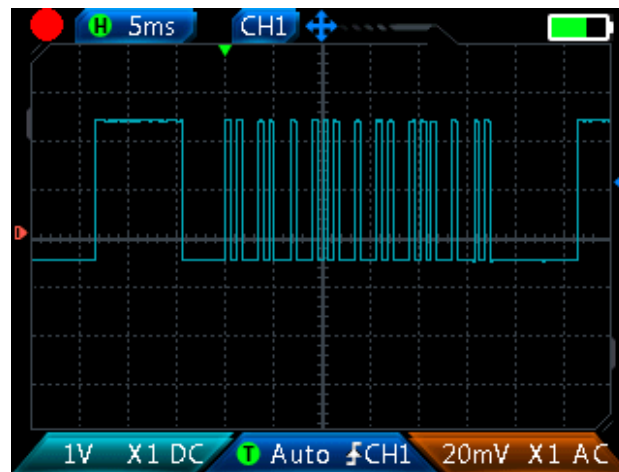


Figure 8 : Example of baseband signal using the standard NEC protocol

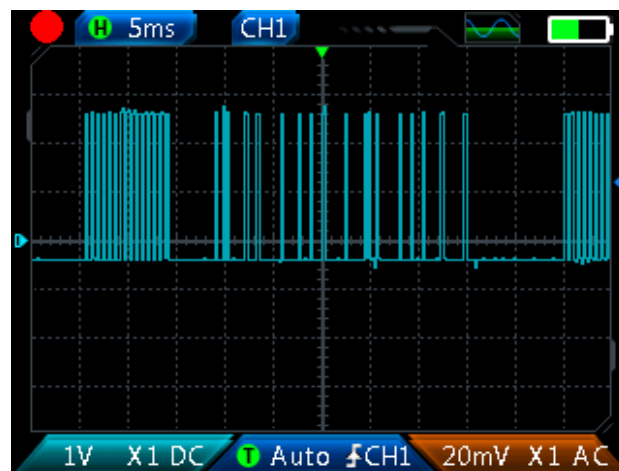


Figure 9 : Example of modulated signal using the standard NEC protocol

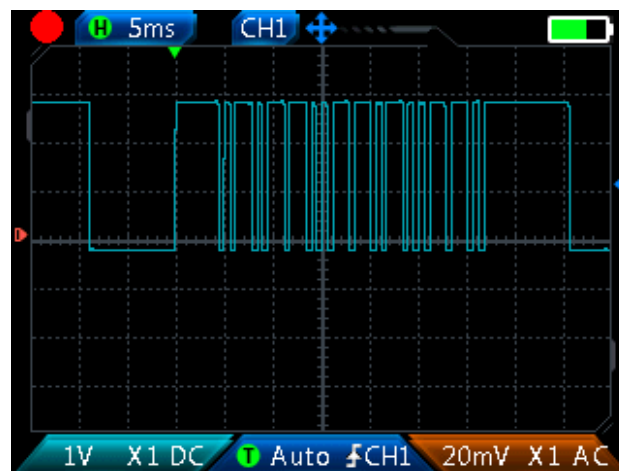


Figure 10 : Example of demodulated signal from TSOP1838 module

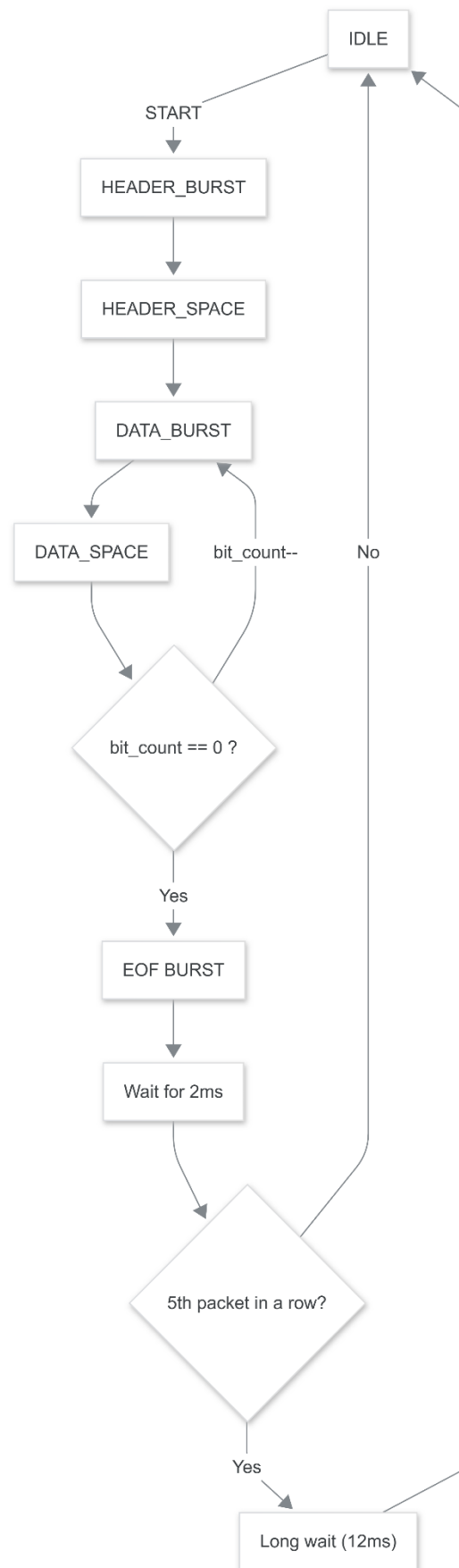


Figure 11 : IR transmitter module state machine flowchart

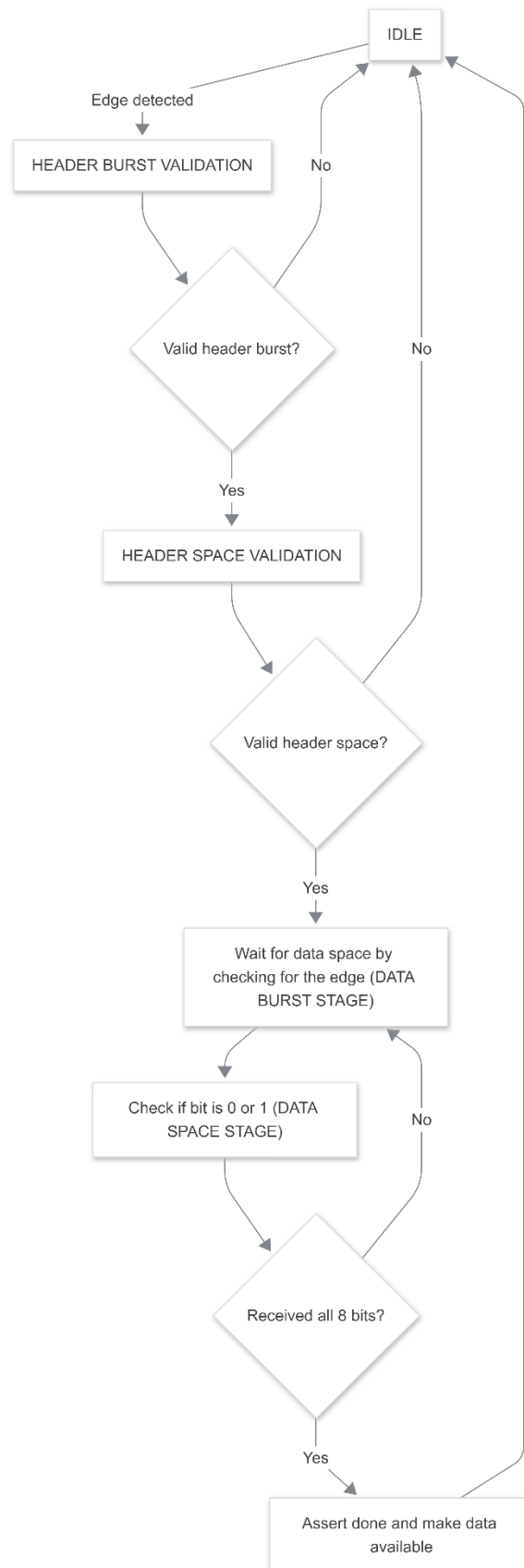


Figure 12 : IR receiver module state machine flowchart

2.5 Search Algorithm

When the target is lost, the search algorithm uses a taxicab spiral to attempt to relocate the target. Each IR packet contains a bit that tells the transmitter whether the laser is on target. The moment the laser is off target, the search function is triggered until the laser hits the target again. The search begins at where the target was lost and has a predefined search area so to keep the search purposeful. The basics of the algorithm in terms of Cartesian coordinates is as follows:

$$+x \rightarrow +y \rightarrow -2x \rightarrow -2y \rightarrow +3x \rightarrow +3y \rightarrow -4x \rightarrow -4y \dots$$

By keeping track of the steps of the motors, we can define soft limits to restrict the range of motion of the transmitter and allows us to home the transmitter. If the transmitter is unable to locate the target after searching the entire area, it will return to the boot up location to wait for the target to be presented manually again.

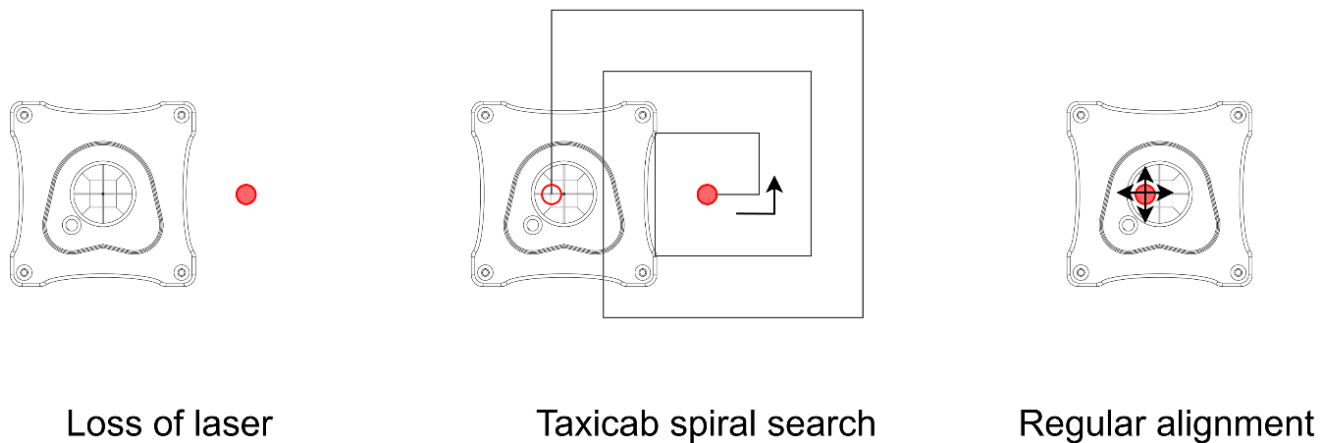


Figure 13 : Search algorithm implementation

The algorithm kicks off when the laser is lost from the view of the QPD. The transmitter will perform a taxicab spiral search to find the target again. Once the target has been located, the laser goes through the regular alignment used to center the laser on the QPD.

3 Challenges

The current configuration of this project is our third iteration. Given the physical nature of our project, the issues we faced involved optics, physics, digital design, and the limits of our components.

3.1 Original Design

Our original design involved a more optics heavy approach. In our proposal for this project, the laser from the transmitter was split at the receiver and reflected back at an offset to the transmitter. This offset was measured by the QPD, which was originally onboard the transmitter.

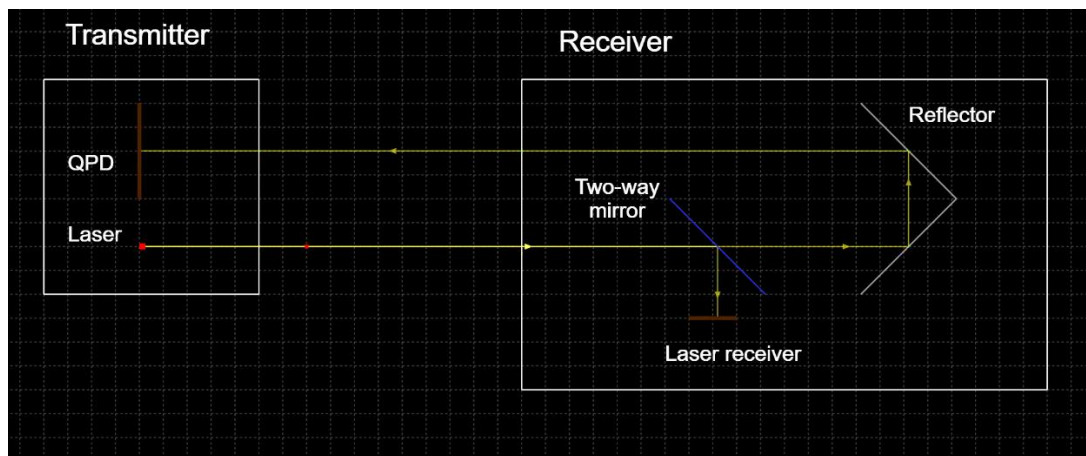


Figure 14 : Original design as presented in project proposal

This design presented multiple challenges, namely in optical physics and practical implementation. The laser we have on hand is a low-power 5mW diode, used primarily in laser pointers. Given this constraint, splitting and reflecting the beam led to losses which were apparent when reading values from the photodiodes, requiring very large gains on the transimpedance amplifier. Further, this original design relied on highly accurate parts and reliable measurements from the QPD.

We also tested a configuration using a retroreflective dot for offset detection. The process followed these steps:

- **Beam Diffusion**
 - The laser was defocused into a cone of light to cover the entire retroreflective dot.
- **Reflection**
 - The dot reflected a fraction of this light back toward the transmitter.
- **Redirection**
 - A beam splitter redirected the returning light toward a QPD.
- **Measurement**
 - The QPD analyzed the reflected light to determine the necessary offset.



Figure 15 : Original transmitter assembly

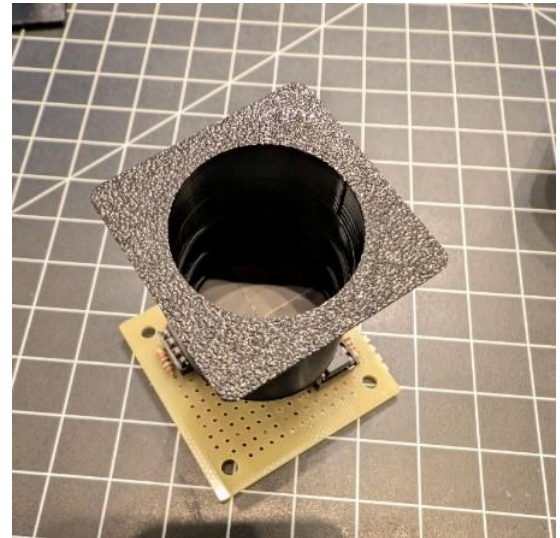


Figure 16 : Close-up of baffle

Above is the original transmitter assembly. On the bottom, we have the laser diode, which goes through a makeshift beam splitter (made from a microscope slide). The reflected light from the retroreflector returns to the transmitter and hits the beam splitter to be reflected into the QPD on the left. The light from the beam splitter goes through a baffle to limit the ambient light and then onto a diffuser, where the light is read by the photodiodes.

The main problems with this configuration lie in the nature of the optics and the limitations of our components. While theoretical demonstrations of this approach seem to work, what we see in practice is that the movement of the laser is practically indiscernible to the QPD. Compounded with other issues we were facing regarding the physics involved, it was decided that we move away from this approach and brainstorm other options.

3.2 Current Design

Our current design does away with reflecting the light back entirely. Instead, the laser from the transmitter directly hits the QPD on the receiver where it is absorbed. The positional data derived from the QPD is sent back via IR to the transmitter for correction.

While this design is more robust than our original idea, it is not free of its fair share of challenges. A common issue between the original and current design stems from the direct drive of the motors used to move the transmitter. The drivers used to run the stepper motors are running at $1/16^{\text{th}}$ of a step. Since a full revolution is 200 full steps, each increment is equivalent to a change of 0.1125° . While at first this sounds like enough resolution, expressing this as an arc length around a two-metre radius yields:

$$\begin{aligned}
 s &= r\theta \\
 &= 2 \text{ m} \cdot \left(\frac{\pi \times 0.1125^{\circ}}{180^{\circ}} \right) \\
 s &= 3.93 \text{ mm}
 \end{aligned}$$

As a ratio of the diameter of the QPD's face ($d = 1$ inch):

$$D = \frac{3.93 \text{ mm}}{25.4 \text{ mm}}$$

$$= 15.5\%$$

This means that approximately, every increment of the stepper moves the laser 15.5% of the diameter of the QPD. While it is indeed possible to stabilize and align the laser at this distance, noticeable jittering can be seen. Since one of our stretch goals for this project is tracking at long range ², this jitter in the movement becomes the limiting factor of the distance we can place the receiver at. To cover the QPD well, a ratio of less than about 25% is ideal as the laser is less likely to step completely off the receiver. Past fifty percent, the laser will be off the receiver in just two steps, leading to an error that is impossible to correct. Below is a graph showing this ratio and the distance from the QPD:

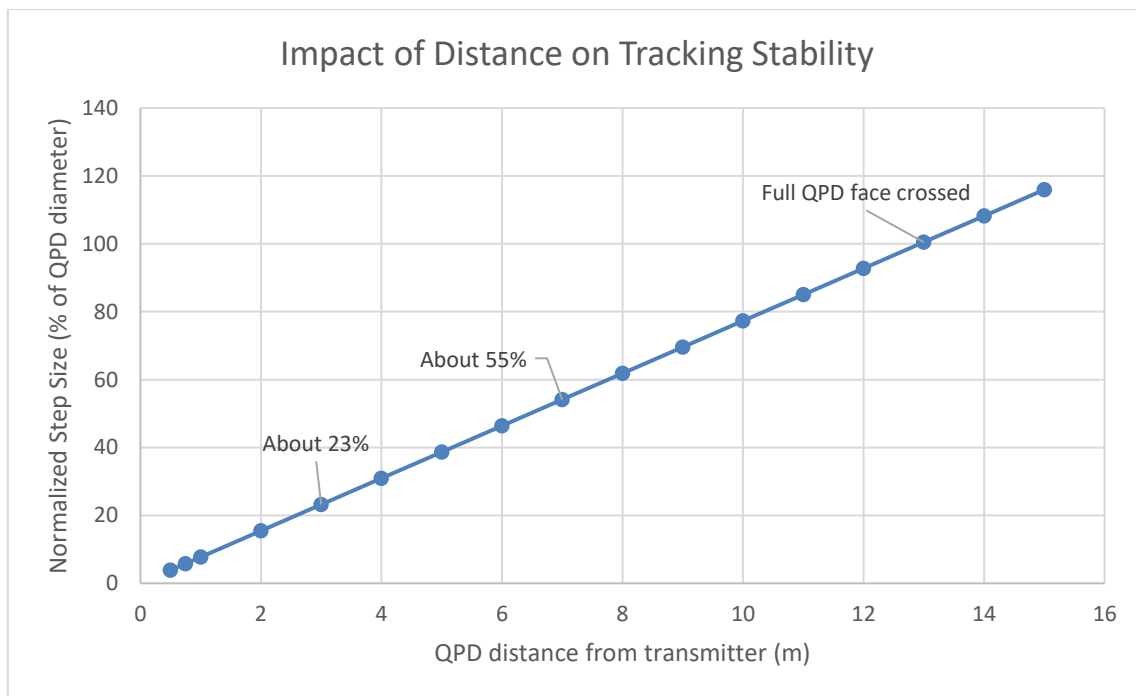


Figure 17 : Impact of Distance on Tracking Stability Graphed

If we push the ratio to 25%, the maximum distance possible will be around three metres. Past four metres, it only takes about three steps to cover the entire face of the QPD which will result in a very jittery alignment. The solution to this problem is to gear the motors such that the angular displacement caused by a step minimizes the arc length and ratio. While this would result in more precise aiming, we would have to account for backlash in 3D printed gearing. To keep the project simple and the robot small, we decided to settle on direct driving as we were already reaching our goals for the range.

² See Introduction for goals

Another important limiting factor to the alignment of the laser on the QPD was the data rate of the infrared transmission. The original NEC IR protocol takes 67.5ms per frame of data, which equates to just short of fifteen updates/packets per second. This was not nearly enough for the speeds we were aiming for. Our fix involved removing error correction and shortening the time needed per frame while considering the timing requirements needed by the IR receiver module. By applying these fixes, we reach a much more reasonable 70 packets per second³.

4 Results and Analysis

To test our system, we placed the transmitter and receiver apart at a distance in a dim room. Each FPGA is configured with a debug system derived from the Lab 4 IP, which is formatted using C code and read on the 'juart-terminal'. This was done so that the system states, the QPD ADC values, and received IR messages could be monitored and evaluated.

Looking at Figure 17, we see a *goldilocks zone* of range: approximately 0.5m to 4m of distance. Knowing this, we started with the lower end: at ~50cm. Per our theory, we should see a more stable alignment at closer distances. This proved to be true to an extent. Due to a minimum distance needed by the IR channel to work effectively, the receiver needs to be at least 15cm from the transmitter. Further, the limited precision of our fixed-point implementation means that the ADC noise was magnified during coordinate calculation, leading to jitter in the output. We tried multiple approaches to correct for the ADC noise including moving average filters and clamps, but they only proved to be mildly successful. What we used instead is a confidence-based hysteresis counter, which is used to better "lock" onto the signal. This leads to less false detections while still preserving the speed of the alignment.

Going up to around mid-range at 2m, we see the system work at its best. While the laser does jitter on the face of the QPD, the transmitter is able to follow the receiver quickly and accurately. The laser remains on the face of the QPD for the entire duration of the tracking, even while moving the receiver quickly.

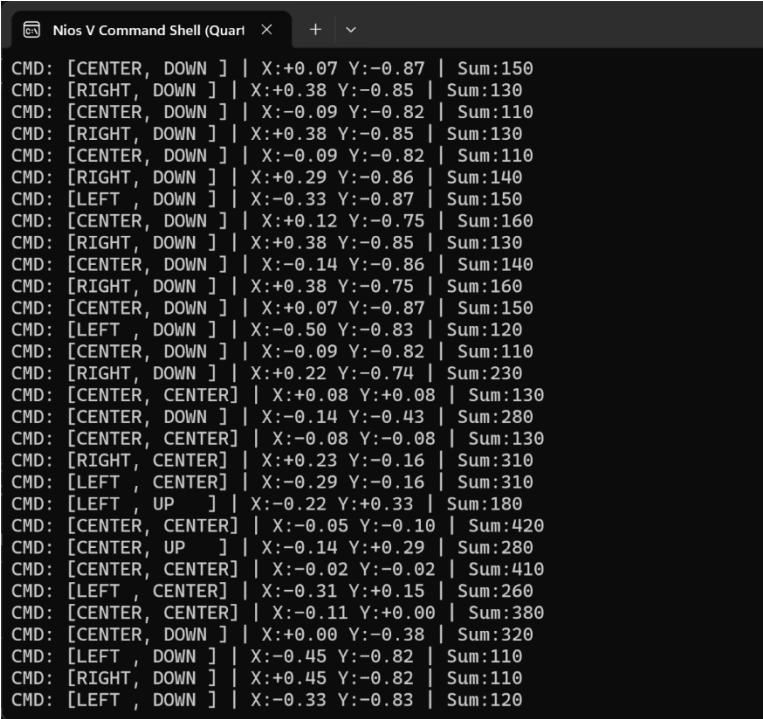


Figure 18 : Receiver during alignment

³ See 6.1 for calculation

Past about 4m, the alignment completely falls apart. The QPD is able to see the laser, but as soon as it tries to perform an alignment, the laser moves off the QPD. As explained in the previous section, this is due to the arc length of a step increasing as the distance goes up. The quality of the laser and its built-in lens also limited our ability to tighten the laser dot at that distance.

These results show us that while the digital hardware implementation successfully leverages the FPGA's parallel processing for high-speed tracking, the system's efficacy is ultimately bound by the geometric relationship between angular resolution and distance. The *goldilocks zone* identifies the sweet spot where our fixed-point coordinate math and stepper resolution intersect perfectly with the sensor's sensitivity. While the arc-length limitation at long ranges (>4m) remains a physical constraint of the stepper increments, the successful stabilization at mid-range confirms that our confidence-based hysteresis counter logic effectively mitigated ADC noise without sacrificing real-time performance. Ultimately, the project demonstrates that an FPGA-centric design provides the low-latency response necessary for autonomous optical alignment, even when faced with the physical limits of our components.



```

CMD: [CENTER, DOWN ] | X:+0.07 Y:-0.87 | Sum:150
CMD: [RIGHT, DOWN ] | X:+0.38 Y:-0.85 | Sum:130
CMD: [CENTER, DOWN ] | X:-0.09 Y:-0.82 | Sum:110
CMD: [RIGHT, DOWN ] | X:+0.38 Y:-0.85 | Sum:130
CMD: [CENTER, DOWN ] | X:-0.09 Y:-0.82 | Sum:110
CMD: [RIGHT, DOWN ] | X:+0.29 Y:-0.86 | Sum:140
CMD: [LEFT, DOWN ] | X:-0.33 Y:-0.87 | Sum:150
CMD: [CENTER, DOWN ] | X:+0.12 Y:-0.75 | Sum:160
CMD: [RIGHT, DOWN ] | X:+0.38 Y:-0.85 | Sum:130
CMD: [CENTER, DOWN ] | X:-0.14 Y:-0.86 | Sum:140
CMD: [RIGHT, DOWN ] | X:+0.38 Y:-0.75 | Sum:160
CMD: [CENTER, DOWN ] | X:+0.07 Y:-0.87 | Sum:150
CMD: [LEFT, DOWN ] | X:-0.50 Y:-0.83 | Sum:120
CMD: [CENTER, DOWN ] | X:-0.09 Y:-0.82 | Sum:110
CMD: [RIGHT, DOWN ] | X:+0.22 Y:-0.74 | Sum:230
CMD: [CENTER, CENTER] | X:+0.08 Y:+0.08 | Sum:130
CMD: [CENTER, DOWN ] | X:-0.14 Y:-0.43 | Sum:280
CMD: [CENTER, CENTER] | X:-0.08 Y:-0.08 | Sum:130
CMD: [RIGHT, CENTER] | X:+0.23 Y:-0.16 | Sum:310
CMD: [LEFT, CENTER] | X:-0.29 Y:-0.16 | Sum:310
CMD: [LEFT, UP ] | X:-0.22 Y:+0.33 | Sum:180
CMD: [CENTER, CENTER] | X:-0.05 Y:-0.10 | Sum:420
CMD: [CENTER, UP ] | X:-0.14 Y:+0.29 | Sum:280
CMD: [CENTER, CENTER] | X:-0.02 Y:-0.02 | Sum:410
CMD: [LEFT, CENTER] | X:-0.31 Y:+0.15 | Sum:260
CMD: [CENTER, CENTER] | X:-0.11 Y:+0.00 | Sum:380
CMD: [CENTER, DOWN ] | X:+0.00 Y:-0.38 | Sum:320
CMD: [LEFT, DOWN ] | X:-0.45 Y:-0.82 | Sum:110
CMD: [RIGHT, DOWN ] | X:+0.45 Y:-0.82 | Sum:110
CMD: [LEFT, DOWN ] | X:-0.33 Y:-0.83 | Sum:120

```

Figure 19 : Example of debug console during testing (receiver side)

Using the C-code to our advantage, we pre-formatted the data before outputting it to the console to make it easier to read. From left to right, the columns tell us what quadrant the QPD sees the laser on, the X and Y error from the center of the QPD, and the sum of the ADC values. The sum is used as a threshold value to determine if what we're seeing is a bright spot in ambient light or the laser.

5 Conclusion

In this project, we built a laser alignment robot that sends correction data over infrared. With everything running on hardware, we were able to make the robot track the target solely through instantaneous ADC readings without any predictions based on velocity and acceleration. By putting the QPD on the receiver and transmitting the offset data back over IR, we demonstrated wireless communication between two separate FPGAs. Further timing optimizations allowed us to increase the rate of data sent over IR, resulting in even faster tracking. Lastly, the search algorithm allowed for quick relocation of the target when recently lost and enabled searching of the target in a larger area.

Some limitations we experienced were the angular resolution of the stepper motors, the quality of the laser, and the accuracy and reliability of the makeshift QPD. With more time and resources, we would have designed gearing for transmitter and purchased a higher quality laser and a real quadrant photodiode.

Of the objectives assigned to us, here is what we achieved:

- ✓ Azimuth (yaw) and elevation (pitch) tracking of a moving retroreflective target using a laser
 - Yaw and pitch tracking completed using stepper motors
 - Target is not retroreflective, but communicates the position of the laser instead
- ✓ Effective tracking/transmission with the target further than 2m and of size less than 10cm
 - Testing of the system was done primarily at 2m and greater
 - Target was only 1 inch in diameter

From the stretch goals, we accomplished:

- ✓ Target acquisition
 - Implemented using the search algorithm described in Section 2.5
- Data communication with the target
 - The target communicates with the transmitter for positional correction
 - Transmission to the target (high-speed laser) not implemented
- Able to acquire/track at long range
 - Depending on the definition of long-range, this was accomplished.
 - The given distance goal was two metres. The testing distance we used in demo was three metres (150%) and the system tracked and acquired well.
- ✗ Able to track target moving at high-speed
 - Given the constraints of our system (described in Section 3.2), the speed is mostly limited by the data rate of our infrared implementation and the angular resolution of the gimbal's steppers

Despite not being able to modulate the laser for data transmission, the project still demonstrated communication through infrared, and we achieved most of the tracking functionality by leveraging the strengths of FPGAs. To be able to track the target moving at high-speed would most likely require very different choices in hardware and a fundamental change in implementation. All things considered, this was highly valuable experience and greatly contributed to our learning.

6 Appendix

6.1 Infrared Transmission

This section discusses the calculation regarding the refresh rate of the system governed by the infrared transmission protocol in use.

Table 1 : Custom IR protocol timings

Use	Duration
Header burst	2ms
Header space	1ms
Data burst Data zero space EOF burst	300μs
Data one space	700μs
Packet spacing	2ms
Group spacing	12ms

Given the fact that 1's and 0's are identified via the duration of a space, packets will be of differing durations. To simplify calculations, we will be using the average timing by transmitting a message of 8'b11110000.

1. Single packet cycle

- Each packet consists of the header, data, end of frame (EOF) and a space between packets. This yields:

$$\begin{aligned}
 t_{packet} &= t_{header} + t_{bits} + t_{EOF} + t_{buffer} \\
 &= (2 \text{ ms} + 1 \text{ ms}) + (8 \times 300 \text{ μs} + 4 \times 300 \text{ μs} + 4 \times 700 \text{ μs}) + 300 \text{ μs} + 2 \text{ ms} \\
 &= 11.7 \text{ ms}
 \end{aligned}$$

2. Full transmission cycle

- A full transmission cycle involves five consequent packets sent, followed by a wait to avoid triggering the active gain control of the receiver module.

$$\begin{aligned}
 t_{full} &= 5 \cdot t_{packet} + t_{group} \\
 &= 5 \times 11.7 \text{ ms} + 12 \text{ ms} \\
 &= 70.5 \text{ ms}
 \end{aligned}$$

3. Packets per second

- We can now calculate how many full cycles fit into one second to calculate the approximate refresh rate of the system.

$$\begin{aligned}
 f &= \frac{1}{t_{full}} \\
 &= \frac{1}{70.5 \times 10^{-3} \text{ s}} \\
 &= 14.18 \text{ full cycles per second}
 \end{aligned}$$

$$\begin{aligned}
 F &= 14.18 \text{ full cycles per second} \times 5 \text{ packets per cycle} \\
 &= 70.9 \text{ packets per second} \\
 &\approx \mathbf{70 \text{ packets per second}}
 \end{aligned}$$

6.2 Infrared Receiver Mechanics

*Note that, for the purposes of consistency, the descriptions here regarding HIGH/LOW and falling/rising edge are from the transmitters point of view. The IR receiver module used in this project is **ACTIVE LOW** and the true values read by the module are in reverse relative to the transmitter.*

Since the IR receiver module used in this project demodulates the signal itself, our code is responsible for three main tasks:

1. Checking if the header is valid
2. Figuring out if a transmitted bit is a one or a zero

Making sure we have a valid header is essential to making sure a received signal is not just noise, which would wreak havoc on the system. The timings are checked within a given threshold of the appropriate duration, which signal our state machine to begin receiving data.

Data transmission works by waiting for the data burst's falling edge and counting the amount of time that the space takes to go HIGH again. We take the midpoint of the space needed to encode a 1 and a 0 and use that as the threshold for a logical 0. If the duration is longer than the threshold, it's a logical 1. If it's shorter, it's a logical 0. This data is put into a shift register and the next bit is received.

At the end of transmission, a 'done' signal is flipped HIGH by the module and the data is available for use by the system.

6.3 Bonus photos

The photos below are included for no particular reason, just for fun. Please enjoy!



Figure 20 : Close-up of transmitter head

In this photo, we see the transmitter head's integral components: the laser diode on the left, and the infrared receiver on the right. The head was designed such that the laser could be focused without taking it apart using the dial on the lens.

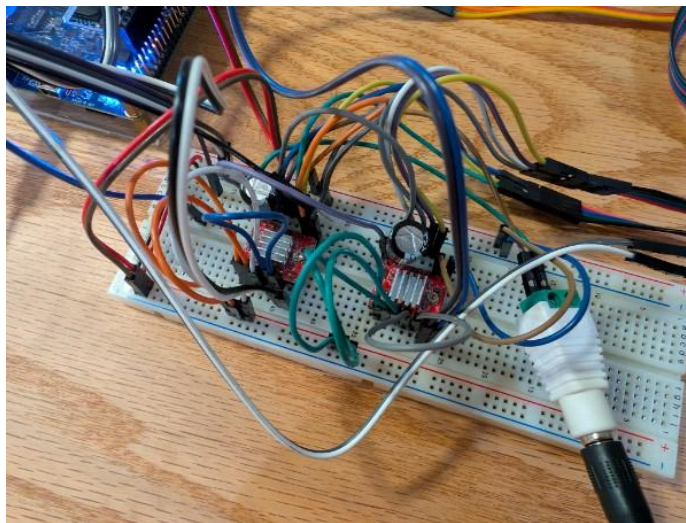


Figure 21 : Motor drivers used for the yaw and pitch steppers

This photo shows the twin motor drivers used by the FPGA to address the yaw and pitch movement of the transmitter gimbal robot. They are powered by a separate external supply to minimize noise in the circuit.



Figure 22 : Original transmitter gimbal robot

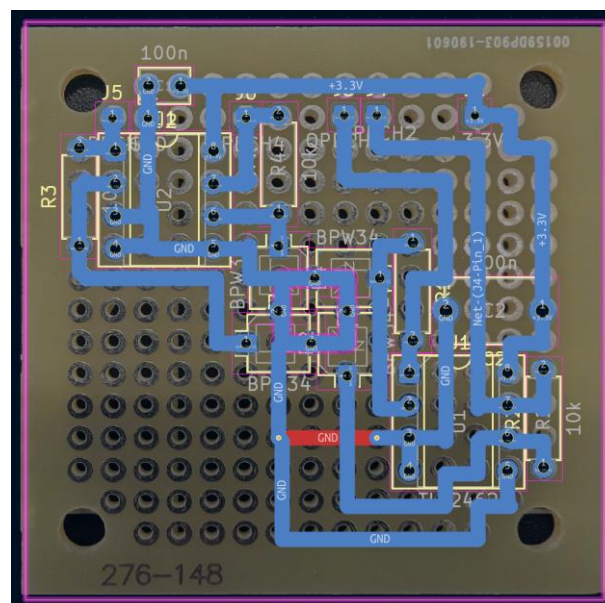


Figure 23 : QPD board planned on KiCad

6.4 Transmitter Code

6.4.1 laser_top.sv

```
// laser_top.sv
// Rafael Banalan 2026-04-23
/*
    This module ties all the sub-modules of the transmitter module, and also
    provides the needed step_tick for the search_controller.
*/
module laser_top
( input logic clk50, // internal system 50MHz clock
  input logic [1:0] KEY, // onboard FPGA dev board buttons
  input logic [1:0] GPIO_1,
  output logic [7:0] LED, // onboard FPGA dev board LED strip
  output logic [3:0] GPIO_0 // motor driver output
  ) ;

    logic cyawstep, cyawdir, cpitchstep, cpitchdir;
    logic rx_done;
    logic [7:0] ir_data;
    logic step_tick;
    logic searching;
    logic on_target;

    assign on_target = ir_data[4];

    logic [19:0] clk_count;
    always_ff @(posedge clk50) begin
        if (clk_count >= 500_000) begin
            clk_count <= 0;
            step_tick <= 1;
        end else begin
            clk_count <= clk_count + 1;
            step_tick <= 0;
        end
    end

    ccom nios_system(
        .clk50_clk(clk50),
        .reset_n_reset_n(KEY[0]),
        .status_export({ 3'b0, searching, ir_data })
    );

    ir_rx r0(
        .clk50(clk50),
        .ir_input(GPIO_1[0]),
        .reset(!KEY[0]),
        .done(rx_done),
        .data(ir_data)
    );

    search_controller controller0 (
        .clk50(clk50),
        .reset(!KEY[0]),
        .return_home(!KEY[1]),
        .on_target(on_target),
        .step_tick(step_tick),
        .rx_done(rx_done),

```



```
.ir_yaw_en(ir_data[0]),
.ir_yaw_dir(!ir_data[1]),
.ir_pitch_en(ir_data[2]),
.ir_pitch_dir(!ir_data[3]),

.out_yaw_en(cyawstep),
.out_yaw_dir(cyawdir),
.out_pitch_en(cpitchstep),
.out_pitch_dir(cpitchdir),
.is_searching(searching)
);

driver simpledriver (
    .clk50(clk50),
    .yaw_step_en(cyawstep),
    .pitch_step_en(cpitchstep),
    .yaw_step(GPIO_0[0]),
    .pitch_step(GPIO_0[2])
);

assign GPIO_0[1] = cyawdir;
assign GPIO_0[3] = cpitchdir;
assign LED = ir_data;

endmodule
```

6.4.2 ir_rx.sv

```
// ir_rx.sv
// Rafael Banalan 2026-04-06 Modified by Rodriguez Chan
/*
    This module uses a modified version of the NEC infrared transmission
    protocol. The main difference is that instead of an address+inv and
    data+inv state, we use one data state of width 8 bits.
*/

// THE RECEIVER IS ACTIVE LOW
// THE RECEIVER IS ACTIVE LOW
// THE RECEIVER IS ACTIVE LOW
// THE RECEIVER IS ACTIVE LOW
// THE RECEIVER IS ACTIVE LOW
// THE RECEIVER IS ACTIVE LOW
// THE RECEIVER IS ACTIVE LOW

module ir_rx(
    input logic clk50,
    input logic ir_input,
    input logic reset,
    output logic recv,
    output logic done,
    output logic [7:0] data
);

    // Timing constants (ticks using 50 MHz period = 20ns)

    localparam BURST_MIN = 80_000;    // 1.6ms
    localparam BURST_MAX = 120_000;   // 2.4ms

    localparam SPACE_MIN = 35_000;    // 0.7ms
    localparam SPACE_MAX = 65_000;    // 1.3ms

    localparam DATA_BURST_MIN = 5_000;    // 100us
    localparam DATA_BURST_MAX = 22_000;   // 440us

    localparam BIT_THRESHOLD = 25_000;    // 500us

    localparam TIMEOUT = 500_000; // 10ms total timeout for recovery

    // Edge detector (metastability protection)
    logic ir_sync0, ir_sync;
    always_ff @( posedge clk50 ) begin
        ir_sync0 <= ir_input;
        ir_sync  <= ir_sync0;
    end

    // Receiver state machine
    typedef enum logic [2:0] {
        IDLE, HEADER_BURST, HEADER_SPACE, DATA_BURST, DATA_SPACE, VALIDATE
    } state_t;

    state_t state;
    logic [19:0] tick_count;
    logic prev_level;
    logic [3:0] bit_count;
    logic [7:0] shift_reg;

    always_ff @( posedge clk50 ) begin
```

```

        if ( reset ) begin
            state <= IDLE;
            tick_count <= 0;
            prev_level <= 1'b1;
            bit_count <= 0;
            shift_reg <= 0;
            done <= 0;
            recv <= 0;
            data <= 0;
        end else begin
            done <= 0;

            if ( ir_sync == prev_level ) begin
                // No edge keep counting
                if ( tick_count < 20'hFFFFFF ) // saturate to avoid
overflow
                    tick_count <= tick_count + 1;
            end else begin
                // Edge detected
                case ( state )
                    IDLE: begin
                        // Wait for falling edge (start of header
burst)

                        if ( prev_level == 1'b1 ) begin
                            recv <= 1;
                            state <= HEADER_BURST;
                        end
                        tick_count <= 0;
                    end

                    HEADER_BURST: begin
                        // Rising edge

                        if ( tick_count >= BURST_MIN && tick_count <=
BURST_MAX ) begin

                            state <= HEADER_SPACE;
                        end else begin
                            recv <= 0;
                            state <= IDLE;
                        end
                        tick_count <= 0;
                    end

                    HEADER_SPACE: begin
                        //Falling edge
                        if ( tick_count >= SPACE_MIN && tick_count <=
SPACE_MAX ) begin

                            bit_count <= 0;
                            shift_reg <= 0;
                            state <= DATA_BURST;
                        end else begin
                            recv <= 0;
                            state <= IDLE;
                        end
                        tick_count <= 0;
                    end

                    DATA_BURST: begin
                        // Rising edge

```

```

                                if ( tick_count >= DATA_BURST_MIN &&
tick_count <= DATA_BURST_MAX ) begin
                                    state <= DATA_SPACE;
                                end else begin
                                    recv <= 0;
                                    state <= IDLE;
                                end
                                tick_count <= 0;
                            end

                            DATA_SPACE: begin
                                // Falling edge: decode bit by HIGH duration
                                shift_reg <= {shift_reg[6:0], (tick_count >
BIT_THRESHOLD)};

                                tick_count <= 0;

                                if ( bit_count == 4'd7 ) begin
                                    state <= VALIDATE;
                                end else begin
                                    bit_count <= bit_count + 1;
                                    state <= DATA_BURST;
                                end
                            end

                            VALIDATE: begin
                                state <= IDLE;
                                recv <= 0;
                            end
                            endcase
                            prev_level <= ir_sync;

                        end

                        // Validate by checking to see if the data matches the inverted
data
                        if ( state == VALIDATE ) begin
                            data <= shift_reg;
                            done <= 1;
                            state <= IDLE;
                            recv <= 0;
                        end

                        // Timeout if we get stuck in a state for too long
                        if ( tick_count >= TIMEOUT && state != IDLE ) begin
                            state <= IDLE;
                            recv <= 0;
                            bit_count <= 0;
                            shift_reg <= 0;
                            tick_count <= 0;
                        end
                    end
                end
            end
        endmodule

```

6.4.3 search_controller.sv

```
// search_controller.sv
// Rodriguez Chan 2026-04-12
/*
    This module handles the high-level functionality of the transmitter unit. It
    does this by switching between the high-level states of the transmitter's
    behaviour, as well as providing the needed logic to perform the search itself.
*/

module search_controller (
    input logic clk50,
    input logic reset,          // KEY[0]
    input logic return_home,    // KEY[1]
    input logic on_target,
    input logic step_tick,
    input logic rx_done,

    input logic ir_yaw_en, ir_yaw_dir,
    input logic ir_pitch_en, ir_pitch_dir,

    output logic out_yaw_en, out_yaw_dir,
    output logic out_pitch_en, out_pitch_dir,
    output logic is_searching
);

    // Constraints
    localparam signed [15:0] BOOT_LIMIT = 266;
    localparam signed [15:0] PITCH_LIMIT = 89;
    localparam signed [15:0] YAW_LIMIT = 177;

    // Search Configuration
    localparam [15:0] SEARCH_MAX_SIDE = 200;
    localparam [31:0] PULSE_WIDTH = 50_000;
    localparam [15:0] DENSITY_STEP = 3;

    typedef enum logic [2:0] {
        BOOT_UP          = 3'd0,
        IDLE_WAIT         = 3'd1,
        TRACKING          = 3'd2,
        SEARCHING         = 3'd3,
        RETURNING         = 3'd4,
        FORCED_RETURN     = 3'd5
    } state_t;

    state_t state;
    logic signed [15:0] pos_x, pos_y;
    logic [15:0] side_limit, step_count;
    logic [1:0] spiral_dir;
    logic [31:0] pulse_timer;
    logic has_seen_target;

    assign is_searching = (state == SEARCHING || state == RETURNING || state ==
FORCED_RETURN);

    always_ff @(posedge clk50 or posedge reset) begin
        if (reset) begin
            state <= BOOT_UP;
            pos_x <= 0; pos_y <= -BOOT_LIMIT;
            has_seen_target <= 0;
            side_limit <= DENSITY_STEP;
        end
    end
endmodule
```

```

    step_count <= 0;
    spiral_dir <= 0;
    pulse_timer <= 0;
end else if (return_home) begin
    state <= FORCED_RETURN;
end else begin

    if (pulse_timer > 0) pulse_timer <= pulse_timer - 1;

    case (state)
        BOOT_UP: begin
            out_pitch_dir <= 1'b1;
            out_pitch_en <= (pulse_timer > 0);
            if (step_tick) begin
                if (pos_y < 0) begin
                    pos_y <= pos_y + 1;
                    pulse_timer <= PULSE_WIDTH;
                end else state <= IDLE_WAIT;
            end
        end
        IDLE_WAIT: begin
            {out_yaw_en, out_pitch_en} <= 2'b00;
            if (on_target) state <= TRACKING;
        end
        TRACKING: begin
            if (on_target) begin
                has_seen_target <= 1'b1;
                if (rx_done) begin
                    pulse_timer <= PULSE_WIDTH;
                    out_yaw_dir <= ir_yaw_dir;
                    out_pitch_dir <= ir_pitch_dir;
                    if (ir_yaw_en) begin
                        if (ir_yaw_dir && pos_x < YAW_LIMIT)
                            pos_x <= pos_x + 1;
                        else if (!ir_yaw_dir && pos_x > -YAW_LIMIT)
                            pos_x <= pos_x - 1;
                    end
                    if (ir_pitch_en) begin
                        if (ir_pitch_dir && pos_y < PITCH_LIMIT)
                            pos_y <= pos_y + 1;
                        else if (!ir_pitch_dir && pos_y > -PITCH_LIMIT)
                            pos_y <= pos_y - 1;
                    end
                end
                out_yaw_en <= (pulse_timer > 0 && ir_yaw_en);
                out_pitch_en <= (pulse_timer > 0 && ir_pitch_en);
            end else if (has_seen_target) begin
                state <= SEARCHING;
                step_count <= 0; spiral_dir <= 0;
                side_limit <= DENSITY_STEP;
                pulse_timer <= 0; // Immediate reset for clean start
            end
        end
        SEARCHING: begin
            if (on_target) state <= TRACKING;
            else if (side_limit > SEARCH_MAX_SIDE) state <= RETURNING;
            else begin

```

```

        case (spiral_dir)
            0: begin out_yaw_dir <= 1;
                    out_yaw_en <= (pulse_timer > 0);
                    out_pitch_en <= 0;

                    end
            1: begin out_pitch_dir <= 1;
                    out_pitch_en <= (pulse_timer > 0);
                    out_yaw_en <= 0;

                    end
            2: begin out_yaw_dir <= 0;
                    out_yaw_en <= (pulse_timer > 0);
                    out_pitch_en <= 0;

                    end
            3: begin out_pitch_dir <= 0;
                    out_pitch_en <= (pulse_timer > 0);
                    out_yaw_en <= 0;

                    end
        endcase

        endcase

        if (step_tick) begin
            pulse_timer <= PULSE_WIDTH;
            case (spiral_dir)
                0: pos_x <= pos_x + 1;
                1: pos_y <= pos_y + 1;
                2: pos_x <= pos_x - 1;
                3: pos_y <= pos_y - 1;
            endcase

            if (step_count >= side_limit - 1) begin
                step_count <= 0;
                spiral_dir <= spiral_dir + 1;
                if (spiral_dir == 1 || spiral_dir == 3)
                    side_limit <= side_limit + DENSITY_STEP;
                end else begin
                    step_count <= step_count + 1;
                end
            end
        end
    end
end

RETURNING: begin
    if (on_target) state <= TRACKING;
    else if (pos_x == 0 && pos_y == 0) begin
        has_seen_target <= 0;
        state <= IDLE_WAIT;
    end else begin
        out_yaw_dir <= (pos_x < 0);
        out_pitch_dir <= (pos_y < 0);
        out_yaw_en <= (pulse_timer > 0 && pos_x != 0);
        out_pitch_en <= (pulse_timer > 0 && pos_y != 0);

        if (step_tick) begin
            pulse_timer <= PULSE_WIDTH;
            if (pos_x > 0) pos_x <= pos_x - 1;
            else if (pos_x < 0) pos_x <= pos_x + 1;
            if (pos_y > 0) pos_y <= pos_y - 1;
            else if (pos_y < 0) pos_y <= pos_y + 1;
        end
    end
end
end
end

```

```

        FORCED_RETURN: begin
            if (pos_x == 0 && pos_y == -BOOT_LIMIT) begin
                has_seen_target <= 0;
                state <= IDLE_WAIT;
            end else begin
                out_yaw_dir    <= (pos_x < 0);
                out_pitch_dir  <= (pos_y > -BOOT_LIMIT) ? 1'b0 : 1'b1;
                out_yaw_en     <= (pulse_timer > 0 && pos_x != 0);
                out_pitch_en   <= (pulse_timer > 0 && pos_y != -
BOOT_LIMIT);

                if (step_tick) begin
                    pulse_timer <= PULSE_WIDTH;
                    if (pos_x > 0) pos_x <= pos_x - 1;
                    else if (pos_x < 0) pos_x <= pos_x + 1;
                    if (pos_y > -BOOT_LIMIT) pos_y <= pos_y - 1;
                    else if (pos_y < -BOOT_LIMIT) pos_y <= pos_y + 1;
                end
            end
        end
    endcase
end
end
endmodule

```

6.4.4 driver.sv

```

// driver.sv
// Rafael Banalan 2026-03-31
/*
    Simple driver module that idles high to apply
    holding torque when the motors aren't moving.
*/
module driver
(
    input logic clk50,
    input logic yaw_step_en, pitch_step_en,
    output logic yaw_step, pitch_step
);
    logic clk0;
    clkdiv #(50_000_000, 1000) c0 (clk50, clk0);

    always_ff @(posedge clk0) begin
        yaw_step    <= yaw_step_en    ? !yaw_step    : 1'b1;
        pitch_step  <= pitch_step_en  ? !pitch_step  : 1'b1;
    end
endmodule

```


6.5 Receiver Code

6.5.1 laser_rx.sv

```
// laser_rx.sv
// Rafael Banalan 2026-04-01
/*
    This is the top level module for the receiver. It
    ties the sub-modules together.
*/

module laser_rx(
    input logic clk50, clk38,
    input logic [1:0] KEY, // onboard FPGA dev board buttons
    output logic [3:0] GPIO_1, // pin 0 used for transmitter

    // onboard FPGA dev board ADC chip
    input logic ADC_SDO,
    output logic ADC_CONVST, ADC_SDI, ADC_SCK
);

logic [1:0] rx_state_status;
logic [23:0] dxdy;
logic [11:0] q0, q1, q2, q3;

ccom_rx_nios_system(
    .clk50_clk(clk50),
    .reset_n_reset_n(KEY[0]),
    .dxdy_export(dxdy),
    .qpdp1_export({ q3, q2 }),
    .qpdp2_export({ q1, q0 }),
    .status_export({ 6'b111111, rx_state_status})
);

adc_qpd_scanner_qpdrd(
    .clk(clk50),
    .reset(!KEY[0]),
    .sdo(ADC_SDO),
    .convst(ADC_CONVST),
    .sck(ADC_SCK),
    .sdi(ADC_SDI),
    .q0(q0),
    .q1(q1),
    .q2(q2),
    .q3(q3)
);

rx_state_r0 (
    .clk50(clk50),
    .reset(!KEY[0]),
    .dxdy(dxdy),
    .status(rx_state_status),
    .ir_out(GPIO_1[0]),
    .q0(q0),
    .q1(q1),
    .q2(q2),
    .q3(q3)
);

endmodule
```

6.5.2 adc_qpd_scanner.sv

```
// adc_qpd_scanner.sv
// Derived from code by Ed Casas (ltc2308.sv)
// Modified for use by Rafael Banalan
/*
    This code addresses the onboard ADC to read the QPD ADC values. It
    outputs each channel as a ready-to-use 12-bit logic value.
*/

module adc_qpd_scanner (
    input logic clk, reset,          // 50 MHz clock
    input logic sdo,                 // serial data from ADC chip
    output logic convst, sck, sdi,   // ADC control signals

    // Direct outputs for motor/logic modules (latest sampled value)
    output logic [11:0] q0, q1, q2, q3,

    // Additional logic signals
    output logic available
);

// ADC states and state duration counter
typedef enum { start, convert, transfer, acquire } state_t;
state_t state, state_next;
logic [6:0] count, count_next;

`define set_next(s,c) begin state_next=(s); \
    count_next=7'(c)-1'b1; end

always_comb begin
    if (reset)
        `set_next(start, 2)
    else if (count)
        `set_next(state, count)

    else
        case(state)
            start:    `set_next(convert, 126)
            convert:  `set_next(transfer, 24)
            transfer: `set_next(acquire, 8)
            acquire:  `set_next(start, 2)
            default:  `set_next(start, 2)
        endcase
end

always_ff @(posedge clk) begin
    state <= state_next;
    count <= count_next;
end

// ADC control signals
always_ff @(posedge clk) begin
    convst <= (state_next == start);
    sck <= (state_next == transfer && !count_next[0]);
end

// cycle through channels
logic [2:0] ch, prevch;
const logic [2:0] firstch = 3'd0;
```

```

const logic [2:0] lastch = 3'd3;

always_ff @(posedge clk) begin
    if (reset) begin
        ch <= firstch;
        prevch <= firstch;
    end else if (state == convert && !count_next) begin
        // move to next quadrant channel
        ch <= (ch == lastch) ? firstch : ch + 1'b1;
        // remember which channel is currently being read
        prevch <= ch;
    end
end

// ADC configuration on SDI
logic [11:0] adccfg;
assign adccfg = { 1'b1, ch[0], ch[2:1], 2'b10, 6'b0 };

always_ff @(posedge clk)
    sdi <= (state_next == transfer) ? adccfg[count_next >> 1] : 1'b0;

// ADC data on SDO
logic [11:0] sample;
always_ff @(posedge clk) begin
    if (state == transfer && !sck)
        sample <= { sample[10:0], sdo };
end

// when a transfer finishes, save the sample to the specific quadrant register
always_ff @(posedge clk) begin
    if (reset) begin
        q0 <= 12'h0; q1 <= 12'h0; q2 <= 12'h0; q3 <= 12'h0;
    end else if (state == transfer && state_next == acquire) begin
        case (prevch)
            3'd0: q0 <= sample;
            3'd1: q1 <= sample;
            3'd2: q2 <= sample;
            3'd3: q3 <= sample;
        endcase
    end
end
endmodule

```

6.5.3 rx_state.sv

```
// rx_state.sv
// Rafael Banalan 2026-04-01 Modified by Rodriguez Chan
/*
    This module formats the IR transmitter message using
    data from the QPD-- specifically from the qpd_state module.
*/

module rx_state (
    input logic clk50,
    input logic reset,
    input logic [11:0] q0, q1, q2, q3,

    output logic [23:0] dxdy,
    output logic [1:0] status,

    output logic ir_out
);
    //logic en_ir;
    logic on_target, micro;
    logic [4:0] move_com;

    logic [19:0] tx_timer;
    logic tx_pulse;

    always_ff @(posedge clk50) begin
        if (tx_timer >= 1_000_000) begin // 50MHz / 1M = 50Hz
            tx_timer <= 0;
            tx_pulse <= 1'b1;
        end else begin
            tx_timer <= tx_timer + 1;
            tx_pulse <= 1'b0;
        end
    end

    qpd_state qpd0 (.*);

    assign status = { on_target, micro };

    ir_tx ir0(
        .clk50(clk50),
        .reset(reset),
        .start(tx_pulse),
        .data({ 3'b0, move_com }),
        .ir_out(ir_out)
    );

endmodule
```

6.5.4 qpd_state.sv

```
// qpd_state.sv
// Rafael Banalan & Rodriguez Chan 2026-04-05
/*
    This module parses the QPD data into a command to be sent
    to the transmitter via IR.
*/
module qpd_state(
    input logic      clk50,
    input logic      reset,
    input logic [11:0] q0, q1, q2, q3,
    output logic      on_target, micro,
    output logic [23:0] dxdy,
    output logic [4:0] move_com
);

    // Optimized Thresholds
    localparam int TARGET_CONFIDENCE_MAX = 500_000;
    localparam int MOVE_THRESH           = 350;
    localparam int MICRO_THRESH          = 150;

    logic signed [15:0] sum;
    logic signed [15:0] dx_num, dy_num;
    logic [23:0] confidence_counter;

    assign sum      = q0 + q1 + q2 + q3;
    assign dx_num   = (q0 + q1) - (q2 + q3);
    assign dy_num   = (q0 + q3) - (q1 + q2);

    // Confidence Logic
    always_ff @(posedge clk50 or posedge reset) begin
        if (reset) begin
            confidence_counter <= 0;
            on_target <= 1'b0;
        end else begin
            if (sum > 300) begin
                if (confidence_counter < TARGET_CONFIDENCE_MAX)
                    confidence_counter <= confidence_counter + 1;

                // Hysteresis: Require 50k cycles to lock, but don't drop
                immediately
                if (confidence_counter > 50_000) on_target <= 1'b1;
            end else begin
                if (confidence_counter > 0)
                    confidence_counter <= confidence_counter - 1;
                else
                    on_target <= 1'b0;
            end
        end
    end

    logic move_x, move_y, dir_x, dir_y;

    always_ff @(posedge clk50) begin
        // Horizontal Movement
        if (dx_num > MOVE_THRESH) begin
            move_x <= 1'b1; dir_x <= 1'b1;
        end else if (dx_num < -MOVE_THRESH) begin
            move_x <= 1'b1; dir_x <= 1'b0;
        end else begin
            move_x <= 1'b0; dir_x <= 1'b0;
        end
    end
end
```

```

        move_x <= 1'b0;
    end

    // Vertical Movement
    if (dy_num > MOVE_THRESH) begin
        move_y <= 1'b1; dir_y <= 1'b1;
    end else if (dy_num < -MOVE_THRESH) begin
        move_y <= 1'b1; dir_y <= 1'b0;
    end else begin
        move_y <= 1'b0;
    end

    // Micro-adjustment (Unused)
    micro <= (abs16(dx_num) < MICRO_THRESH && abs16(dy_num) < MICRO_THRESH);
end

function automatic signed [15:0] abs16(input signed [15:0] val);
    return (val[15]) ? -val : val;
endfunction

// Debugging output
assign dxdy = { 2'b0, dx_num[15], dx_num[8:0], 2'b0, dy_num[15],
dy_num[8:0] };
assign move_com = { on_target, dir_y, move_y, dir_x, move_x };

endmodule

```

6.5.5 ir_tx.sv

```
// ir_tx.sv
// Rafael Banalan 2026-04-01 Modified by Rodriguez Chan
/*
    This module uses a modified version of the NEC infrared transmission
    protocol. The main difference is that instead of an address+inv and
    data+inv state, we use one data state of width 8 bits.
*/

module ir_tx(
    input logic clk50,

    input logic reset,
    input logic start,
    output logic done,

    input logic [7:0] data,

    output logic ir_out
);

    // Timing constants for 50MHz clock (1 tick = 20ns)
    localparam BIT_COUNT_RESET_VAL = 3'b111;
    localparam TICK_HEADER_BURST = 100_000; // header burst time
    localparam TICK_HEADER_SPACE = 50_000; // header space time
    localparam TICK_DATA_BURST = 15_000; // data burst / data zero / EOF burst
    localparam TICK_DATA_SPACE = 35_000; // data one
    localparam BURST_LIMIT = 4'd5; // sending 5 packets then rest
    localparam TICK_WAIT_GAP = 100_000; // 2ms gap between packets in a burst
    localparam TICK_LONG_WAIT = 600_000; // 12ms AGC recovery gap

    // Generate carrier frequency
    logic carrier;
    clkdiv #(50_000_000, 38_000) c0 (clk50, carrier);

    // Transmitter protocol state machine
    typedef enum logic [3:0] {
        IDLE = 4'd0,
        HEADER_BURST = 4'd1,
        HEADER_SPACE = 4'd2,
        DATA_BURST = 4'd3,
        DATA_SPACE = 4'd4,
        EOF = 4'd5,
        WAIT = 4'd6,
        LONG_WAIT = 4'd7
    } state_t;
    state_t state;

    task next_bit_logic;
        tick_count <= 0;
        if (bit_count == 0) begin
            state <= EOF;
        end else begin
            bit_count <= bit_count - 1'b1;
            state <= DATA_BURST;
        end
    endtask

    logic [7:0] data_out; // A copy of the data to be written, set at start

    logic message; // The signal that will be modulated with the carrier
```

```

logic [2:0] bit_count; // Stores the bit position

logic [19:0] tick_count;

logic [2:0] burst_counter; // Counts 0 to 6

always_ff @(posedge clk50) begin
    if ( reset ) begin
        state <= IDLE;
        tick_count <= 0;
        done <= 0;
    end else case (state)
    default : state <= IDLE;

    // IDLE STATE
    IDLE : begin
        message <= 0;
        bit_count <= BIT_COUNT_RESET_VAL;
        if ( start ) begin
            data_out <= data;
            done <= 0;
            tick_count <= 0;
            state <= HEADER_BURST;
        end
    end

    // HEADER_BURST STATE
    HEADER_BURST : begin
        message <= 1;
        if ( tick_count >= TICK_HEADER_BURST ) begin
            tick_count <= 0;
            state <= HEADER_SPACE;
        end else tick_count <= tick_count + 1'b1;
    end

    // HEADER_SPACE STATE
    HEADER_SPACE : begin
        message <= 0;
        if ( tick_count >= TICK_HEADER_SPACE ) begin
            tick_count <= 0;
            state <= DATA_BURST;
        end else tick_count <= tick_count + 1'b1;
    end

    // DATA BURST
    DATA_BURST : begin
        message <= 1;
        if ( tick_count >= TICK_DATA_BURST ) begin
            tick_count <= 0;
            state <= DATA_SPACE;
        end else tick_count <= tick_count + 1'b1;
    end

    // DATA SPACE
    DATA_SPACE : begin
        message <= 0;
        if ( data_out[bit_count] == 1'b1 ) begin
            if (tick_count >= TICK_DATA_SPACE ) begin
                tick_count <= 0;
            end
        end
    end
end

```



```

        next_bit_logic();
    end else tick_count <= tick_count + 1'b1;
end else begin // if the current bit is a 0
    if ( tick_count >= TICK_DATA_BURST ) begin
        tick_count <= 0;
        next_bit_logic();
    end else tick_count <= tick_count + 1'b1;
end
end

//      // DATA BURST INVERTED
//      DATA_BURST_INV : begin
//          message <= 1;
//          if ( tick_count >= TICK_DATA_BURST ) begin
//              tick_count <= 0;
//              state <= DATA_SPACE_INV;
//          end else tick_count <= tick_count + 1'b1;
//      end
//
//      // DATA SPACE INVERTED
//      DATA_SPACE_INV : begin
//          message <= 0;
//          if( ~data_out[bit_count] == 1'b1 ) begin
//              if (tick_count >= TICK_DATA_SPACE ) begin
//                  tick_count <= 0;
//                  next_bit_logic();
//              end else tick_count <= tick_count + 1'b1;
//          end else begin // if the inverse of the current bit is a 1
//              if ( tick_count >= TICK_DATA_BURST ) begin
//                  tick_count <= 0;
//                  next_bit_logic();
//              end else tick_count <= tick_count + 1'b1;
//          end
//      end
//
//      // EOF STATE
//      EOF : begin
//          message <= 1;
//          if ( tick_count >= TICK_DATA_BURST ) begin
//              tick_count <= 0;
//              done <= 1;
//              burst_counter <= burst_counter + 1'b1;
//              state <= WAIT;
//          end else tick_count <= tick_count + 1'b1;
//      end
//
//      // INTER-PACKET WAIT
//      WAIT : begin
//          message <= 0;
//          done <= 0;
//          if ( tick_count >= TICK_WAIT_GAP ) begin
//              tick_count <= 0;
//              // Increased sprint to 10 packets
//              if ( burst_counter >= BURST_LIMIT ) begin
//                  state <= LONG_WAIT;
//              end else begin
//                  state <= IDLE;
//              end
//          end else tick_count <= tick_count + 1'b1;
//      end
end

```

```
        // AGC RECOVERY WAIT
        LONG_WAIT : begin
            message <= 0;
            if ( tick_count >= TICK_LONG_WAIT ) begin
                tick_count <= 0;
                burst_counter <= 0; // Reset burst credit
                state <= IDLE;
            end else tick_count <= tick_count + 1'b1;
            end
        endcase
    end

    assign ir_out = carrier & message;

endmodule
```

7 References

- [1] Q. Li, S. Xu, J. Yu, L. Yan and Y. Huang, "An improved method for the position detection of a quadrant detector for Free Space Optical Communication," *Sensors*, vol. 19, no. 1, p. 175, 2019.
- [2] SB-Projects, "SB-Projects," 17 July 2025. [Online]. Available: <https://www.sbprojects.net/knowledge/ir/nec.php>. [Accessed 31 March 2026].
- [3] Vishay Semiconductors, "Photo Modules for PCM Remote Control Systems," 13 September 2000. [Online]. Available: https://cdn.velleman.eu/downloads/29/infosheets/vmp502_tsop1838.pdf. [Accessed 31 March 2026].